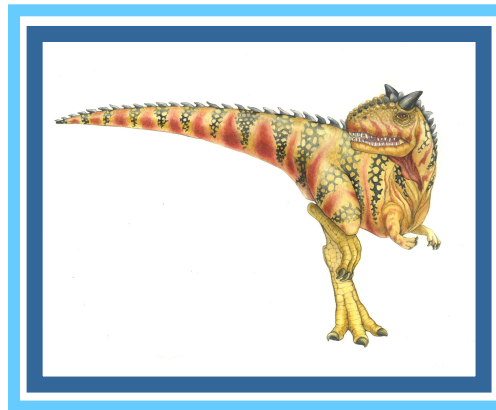


Main Memory





Memory Hierarchy

- Memory hierarchy is arranging different kinds of storage present on a computing device based on speed of access.
- Separates the computer storage based on hierarchy.

Registers

Registers are present inside the CPU. As they are present inside the CPU, they have least access time. Registers are most expensive and smallest in size generally in kilobytes.

Cache memory

Used to store the segments of a program that are frequently accessed by the processor. It is expensive and smaller in size generally in Megabytes

Main memory

Directly communicates with the CPU and with auxiliary memory devices through an I/O processor. Main memory is less expensive than cache memory and larger in size generally in Gigabytes.





Memory Hierarchy

Secondary memory

Secondary memory like magnetic disk are present here. They are used as permanent storage. They are cheaper than main memory and larger in size generally in a few TB.

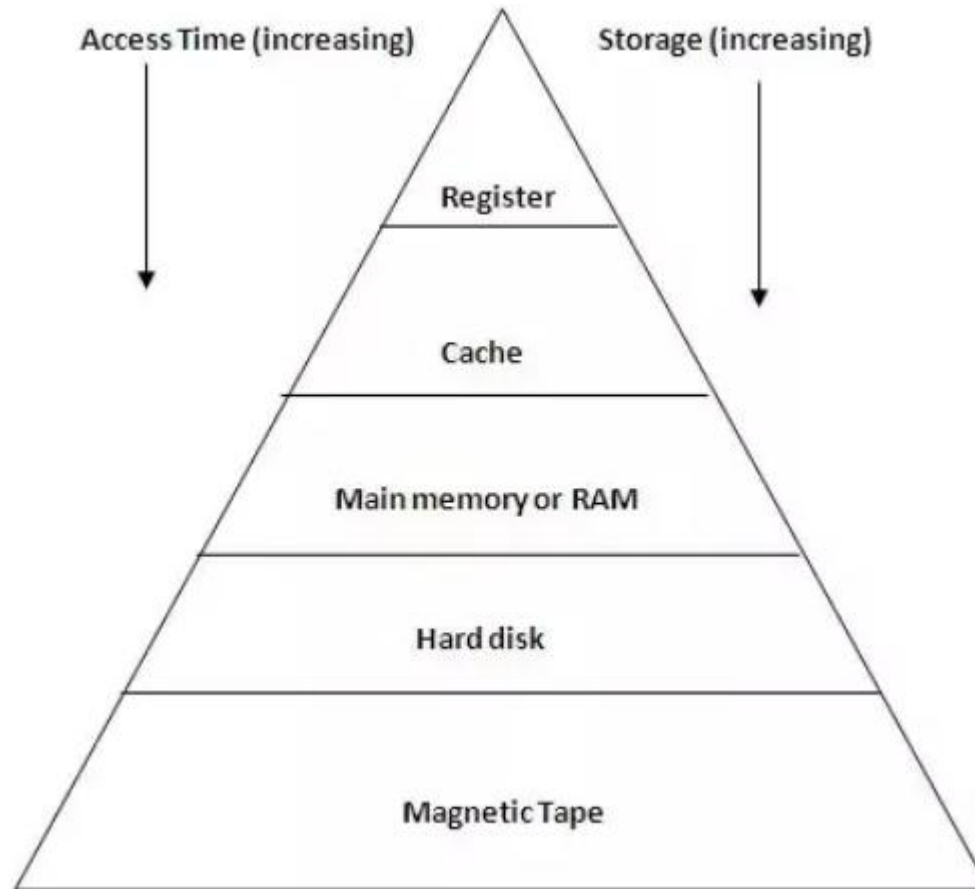
Tertiary memory

Tertiary storage devices like magnetic tape are present at this level. They are used to store removable files and are the cheapest and largest in size (1-20 TB).





Memory Hierarchy





Background

- Memory consists of a large array of words or bytes, each with its own address. The CPU fetches instructions from memory according to the value of the program counter
- After the instruction has been executed on the operands, results may be stored back in memory. The memory unit sees only a stream of memory addresses; it does not know how they are generated
- Memory unit only sees a stream of:
 - addresses + read requests, or
 - address + data and write requests
- Protection of memory required to ensure correct operation





Logical Address

- **Logical Address (Virtual Address)**
- A **logical address** (also called a **virtual address**) is the address **generated by the CPU** during program execution.
- It's the address that a **process thinks** it's using.
- Each process has its **own logical address space**, usually starting at 0.
- **Example:**
If a program accesses memory location 100, that's a *logical address* — it's not yet tied to a real RAM address.





- `int x = 10;`
- `int *p = &x;`
- `printf("%p", p);`
- `p` stores the **address of `x`**, but this address is a **logical (virtual) address**.
- The OS and MMU translate this logical address → to the **physical address** where `x` actually resides in RAM.
- So, the value printed by `printf("%p", p);` is a **virtual address**, meaningful **only within your process** — not the real hardware address.





Physical Address

- **Physical Address**
- The **physical address** is the **actual location in RAM** where the data or instruction resides.
- The **Memory Management Unit (MMU)** or **address binding mechanism** translates the logical address into a physical address.
- **Example:**
If the MMU maps logical address 100 to physical address 3100, then the data at **RAM[3100]** is accessed.





Base Register

- **Base Register**
- The **base register** holds the **starting physical address** of a process's memory area in RAM.
- It is used by the **MMU** to **translate logical** → **physical addresses** dynamically.
- **Example:**
If base register = 3000 and logical address = 100
Physical address = 3100





Limit Register

- **Limit Register**
- The **limit register** holds the **size (range)** of the process's address space.
- It specifies the **maximum valid logical address** a process can generate.

The MMU checks:

If (Logical Address < Limit Register) [within the range]

Access is valid

Else

Trap → Access is invalid, and a trap is generated by the hardware to the operating system's kernel

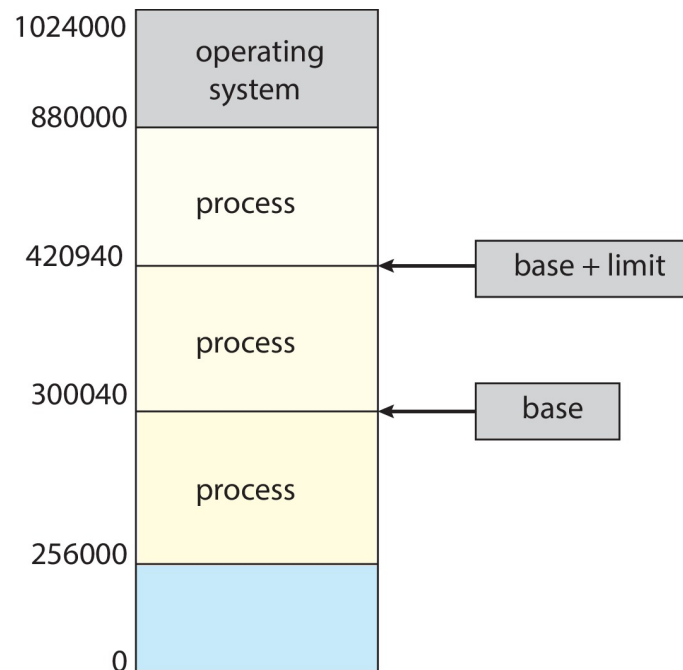
- **Example:**
If limit register = 1200
Process can legally access logical addresses 0–1199 only.





Protection

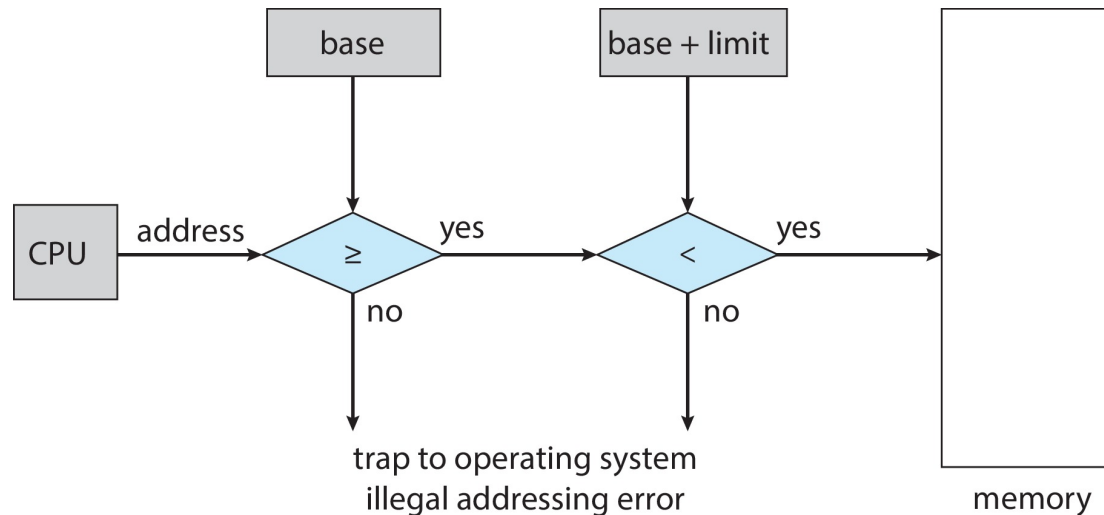
- Need to ensure that a process can access only those addresses in its address space.
- We can provide this protection by using a pair of **base** and **limit registers** which define the logical address space of a process





Hardware Address Protection

- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user





Address Binding

- A program resides on a disk as a binary executable file (a file that contains a **program in machine code** — instructions that the **CPU can execute directly**).
- The program must be brought into memory and placed within a process for it to be executed.
- Depending on the memory management in use, the process may be moved between disk and memory during its execution.
- The collection of processes on the disk that is waiting to be brought into memory for execution forms the **input queue**.
- Normal procedure is to select one of the processes in the input queue and to load that process into memory.
- As the process is executed, it accesses instructions and data from memory.
- Eventually, the process terminates, and its memory space is declared available.





Address Binding

- Addresses represented in different ways at different stages of a program's life
 - Source code addresses usually symbolic addresses
 - Compiled code addresses **bind** to relocatable addresses
 - ▶ i.e., “14 bytes from beginning of this module”
 - Linker or loader will bind relocatable addresses to absolute addresses
 - ▶ i.e., 74014
 - Each binding maps one address space to another





Address binding

❖ Source Code Stage (Symbolic Addresses)

- Written by the programmer.
- Uses **variable names** or **labels** instead of actual memory locations.
- Example:
 - `int total = a + b;`
 - Here total, a, and b are **symbolic addresses**.

❖ Compilation Stage (Relocatable Addresses)

- The **compiler** translates source code into **machine instructions**.
- It doesn't yet know **where in memory** the program will be loaded.
- So, it assigns **relocatable (relative) addresses**, like:
 - "variable x is at offset 14 bytes from start of code segment"
- These addresses are **relative**, not fixed.





Address binding

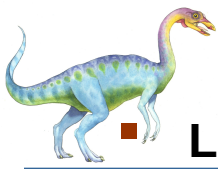
- ❖ **Linking and Loading Stage (Absolute Address/ Actual Address)**
 - The **linker** combines multiple object files (modules) and **assigns absolute addresses**.
 - The **loader** then places the entire program into **main memory**, starting at a chosen base address.
 - For example:
 - The program is loaded starting at address **74000 (base register)**
 - So, the relocatable address **+14** becomes **74014 (actual address)**





- Source Code (.c/.cpp)
 ↓ (compiled)
- Object Code (.o/.obj)
 ↓ (linked)
- Executable File (.exe/.out)
 ↓ (loaded)
- Program in Memory (ready to run)





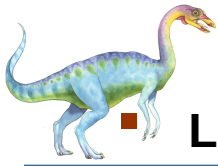
- **Linking** is the process of **combining one or more object files** (and libraries) into a **single executable file**, and **resolving all symbol references** (functions, variables, etc.) among them.
- When you compile a program, each source file is compiled **separately** into an **object file**.
But functions and variables can be defined in **different files**.

```
// file1.c
void hello();
int main() {
    hello(); // Function call
              //from another file
    return 0;}
```

```
// file2.c
#include <stdio.h>
void hello() {
    printf("Hello, world!\n");
}
```

- After compilation:
- file1.o has an **unresolved reference** to hello()
- file2.o has the **definition** of hello()
- The **linker** connects them together into one **executable file** (like program.exe).





■ **Loading** is the process of **placing an executable program into main memory (RAM)** so that it can be executed by the CPU.

- The **loader** is part of the **operating system**.
It performs these steps:
- **Reads the executable file** (output from linker)
- **Allocates memory** space for:
 - Code segment (instructions)
 - Data segment (variables)
 - Stack and heap
- **Loads program segments** into their respective memory areas
- **Sets up registers**:
 - Base and limit registers
 - Stack pointer
 - Instruction counter (to point to main())
- **Transfers control to the program** — CPU starts executing!



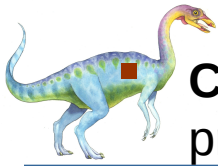


Binding of Instructions and Data to Memory

- Address binding is the process of mapping from one address space to another address space
- Address binding of instructions and data to memory addresses can happen at three different stages

Type	When It Happens	Who Does It	Example
Compile-Time Binding	During compilation	Compiler	Fixed memory location decided by compiler
Load-Time Binding	When program is loaded into memory	Loader	Loader decides base address
Execution-Time Binding	While program is running	MMU (hardware)	Logical → physical address translation at runtime





Compile-time binding - The compiler knows exactly where the program will be loaded in memory. It generates absolute addresses directly in the machine code. When the program executes, those addresses directly refer to physical memory locations.

- Eg : Compiler assumes program starts at 1000.
Instruction uses address 1014 directly in the code.
CPU accesses physical memory 1014 directly.
- **Load-time binding** - The compiler produces relocatable addresses (like +14). The loader decides where the program will be placed (say, at address 4000). Loader adds base address to every relocatable address to make them absolute before execution.
- Eg: Relocatable address = +14
Program loaded at 4000
Loader binds: $4000 + 14 = 4014$
The instruction uses physical address 4014 directly.



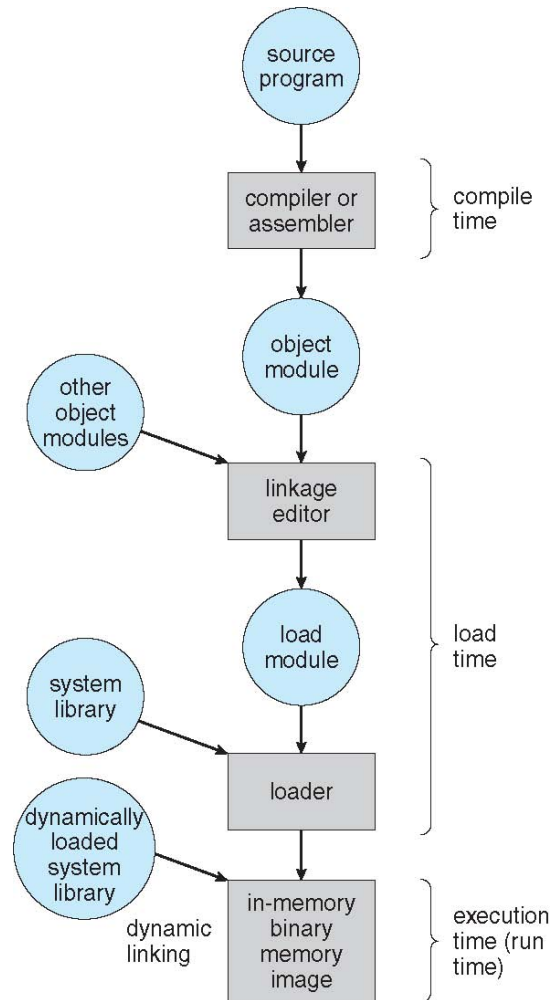


- **Execution-time binding** - The compiler and loader don't know where the program will be in memory. CPU generates logical (virtual) addresses during execution. The MMU (Memory Management Unit) translates them into physical addresses on the fly.
- Eg: CPU generates logical address = 14
- MMU base = 4000
- Physical address = $4000 + 14 = 4014$





Multistep Processing of a User Program





➤ Preprocessing

- Performed by the **preprocessor**
- Tasks:
 - Expand **macros** (#define PI 3.14)
 - Include **header files** (#include <stdio.h>)
 - Remove **comments**
- Input: program.c
- Output: Expanded source code (still text)





➤ **Compilation**

- Performed by the **compiler**.
- Converts the **high-level language** → **assembly or machine code**.
- Checks for **syntax and semantic errors**.
- Produces **object code** (machine instructions with **relocatable addresses**).

- Example:

```
int sum = a + b;
```

- Becomes something like

```
MOV AX, a
```

```
ADD AX, b
```

```
MOV sum, AX
```





➤ Linking

- Performed by the **linker**.
- Combines:
 - Multiple object files
 - Standard libraries (like printf in C)
- Resolves **external references** (functions, variables declared in other files).
- Produces a **single executable file** (e.g., a.exe).
- ◆ Address binding here:
- Converts **relocatable addresses** → **absolute addresses**.





➤ Loading

- Performed by the **loader**, part of the **operating system**.
- Loads the executable into **main memory (RAM)**.
- Allocates memory for:
 - **Code segment**
 - **Data segment**
 - **Stack**
 - **Heap**
- Initializes **registers** and **program counter (PC)** to start execution.
- Finally, it becomes a **process** running on the CPU.





Logical vs. Physical Address Space

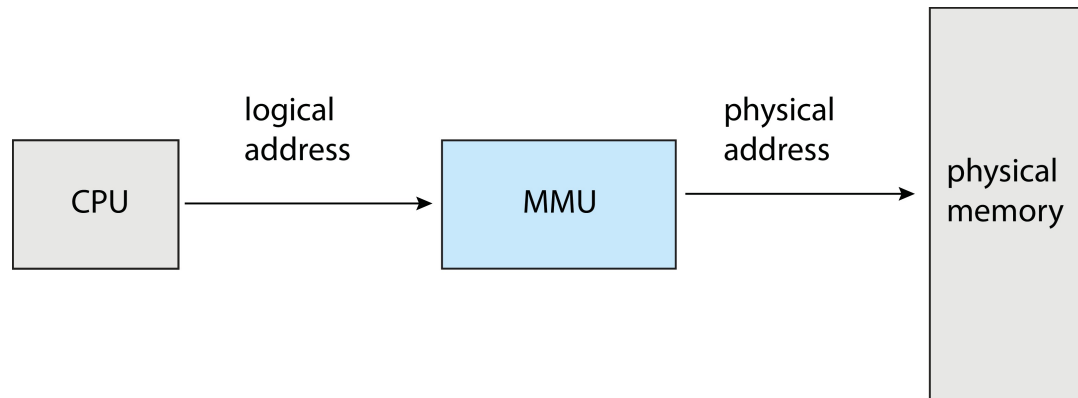
- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
 - **Logical address** – generated by the CPU; also referred to as **virtual address**
 - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program





Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address

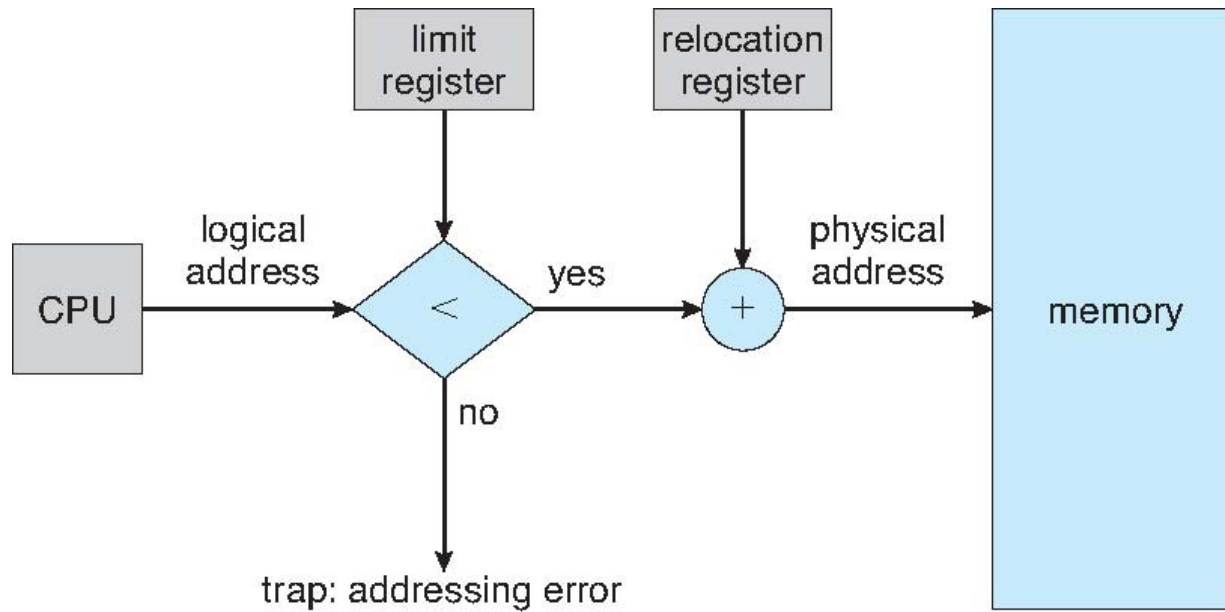




Memory-Management Unit (Cont.)

- Consider simple scheme. which is a generalization of the base-register scheme.
- The base register now called **relocation register**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
 - Execution-time binding occurs when reference is made to location in memory
 - Logical address bound to physical addresses



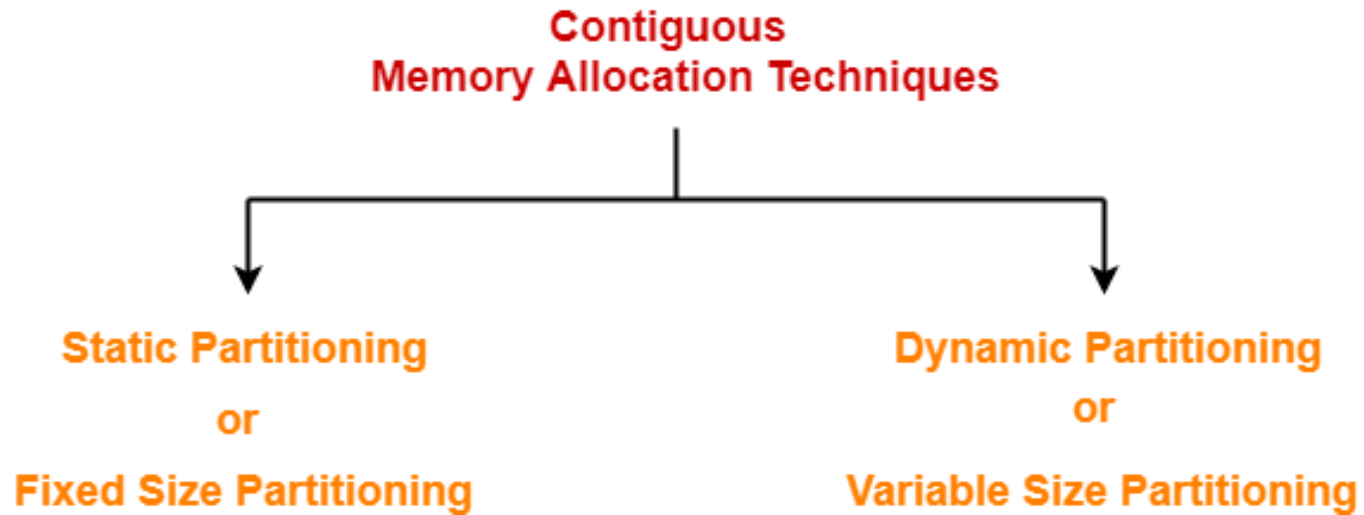




Contiguous Allocation

- Contiguous memory allocation means that each process is allocated a single continuous block of memory in the main memory (RAM).
- In other words, all the bytes or words assigned to a process are stored next to each other in memory — no gaps.
- Main memory is divided into **two parts**:
 - **Operating System (OS)** — usually in lower memory.
 - **User processes** — loaded into the remaining part.
- When a process is loaded, it is given **one contiguous block** large enough to hold:
 - ✓ Its code,
 - ✓ Data, and
 - ✓ Stack.
- Contiguous memory allocation is a memory allocation technique.
- It allows to store the process only in a contiguous fashion.
- Thus, entire process has to be stored as a single entity at one place inside the memory.

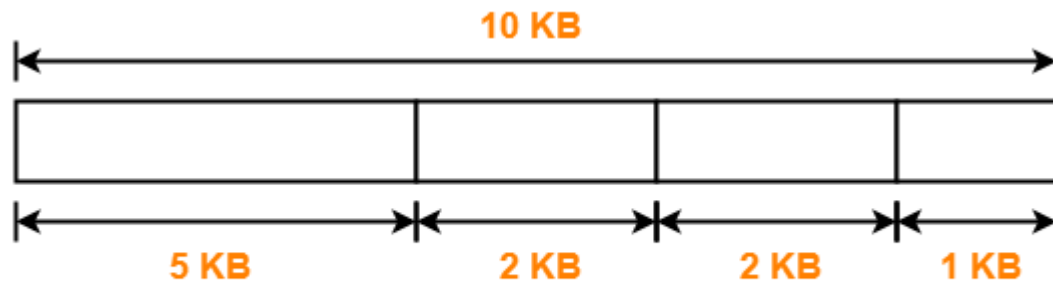






Static Partitioning

- Static Partitioning-
- Static partitioning is a fixed size partitioning scheme.
- In this technique, main memory is pre-divided into fixed size partitions.
- The size of each partition is fixed and can not be changed.
- Each partition is allowed to store only one process.



Memory

- These partitions are allocated to the processes as they arrive.
- The partition allocated to the arrived process depends on the algorithm followed.





Static Partitioning

- Popular algorithms used for allocating the partitions to the arriving processes are
 - ✓ **First Fit Algorithm** –
 - This algorithm starts scanning the partitions serially from the starting.
 - When an empty partition that is big enough to store the process is found, it is allocated to the process.
 - Obviously, the partition size has to be greater than or at least equal to the process size.
 - ✓ **Best Fit Algorithm** –
 - This algorithm first scans all the empty partitions.
 - It then allocates the smallest size partition to the process.
 - ✓ **Worst Fit Algorithm** –
 - This algorithm first scans all the empty partitions.
 - It then allocates the largest size partition to the process.





Static Partitioning

- **For static partitioning,**
- Best Fit Algorithm works best.
- This is because space left after the allocation inside the partition is of very small size.
- Thus, internal fragmentation is least.
- Worst Fit Algorithm works worst.
- This is because space left after the allocation inside the partition is of very large size.
- Thus, internal fragmentation is maximum.





Dynamic Partitioning

- Dynamic partitioning is a variable size partitioning scheme.
- It performs the allocation dynamically.
- When a process arrives, a partition of size equal to the size of process is created.
- Then, that partition is allocated to the process.
- The processes arrive and leave the main memory.
- As a result, free memory areas(holes) of different size are created in the main memory.
- These holes are allocated to the processes that arrive in future.
- Partition allocation algorithms are used to decide which hole should be allocated to the arrived process.

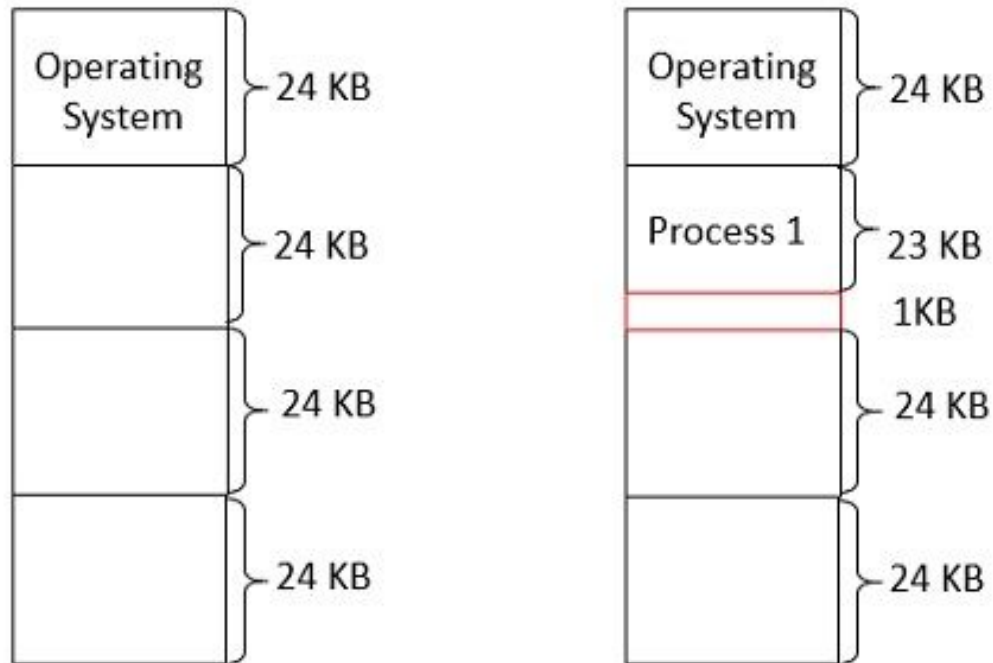




Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used



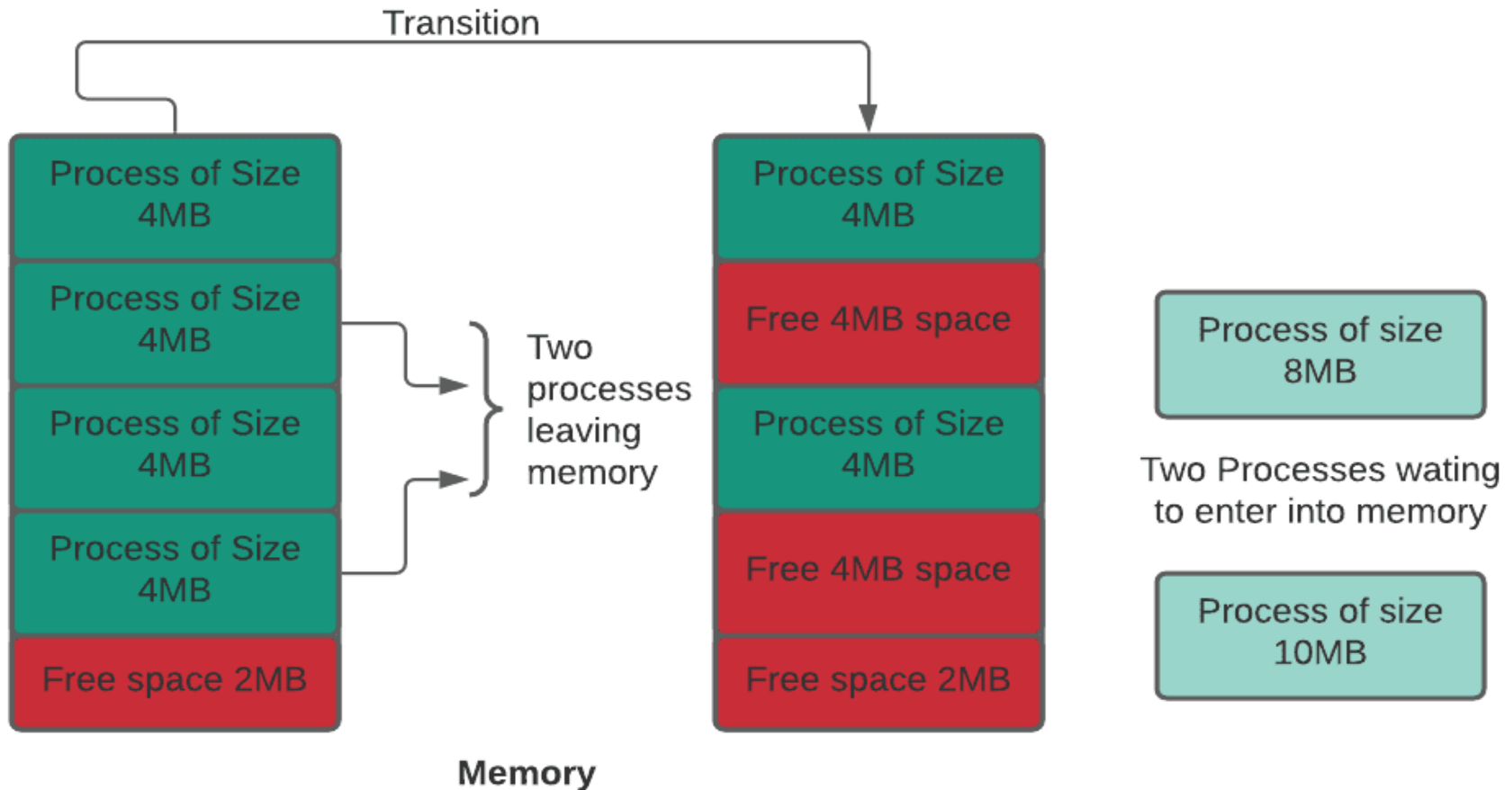


Example of Internal Fragmentation





Example of External Fragmentation





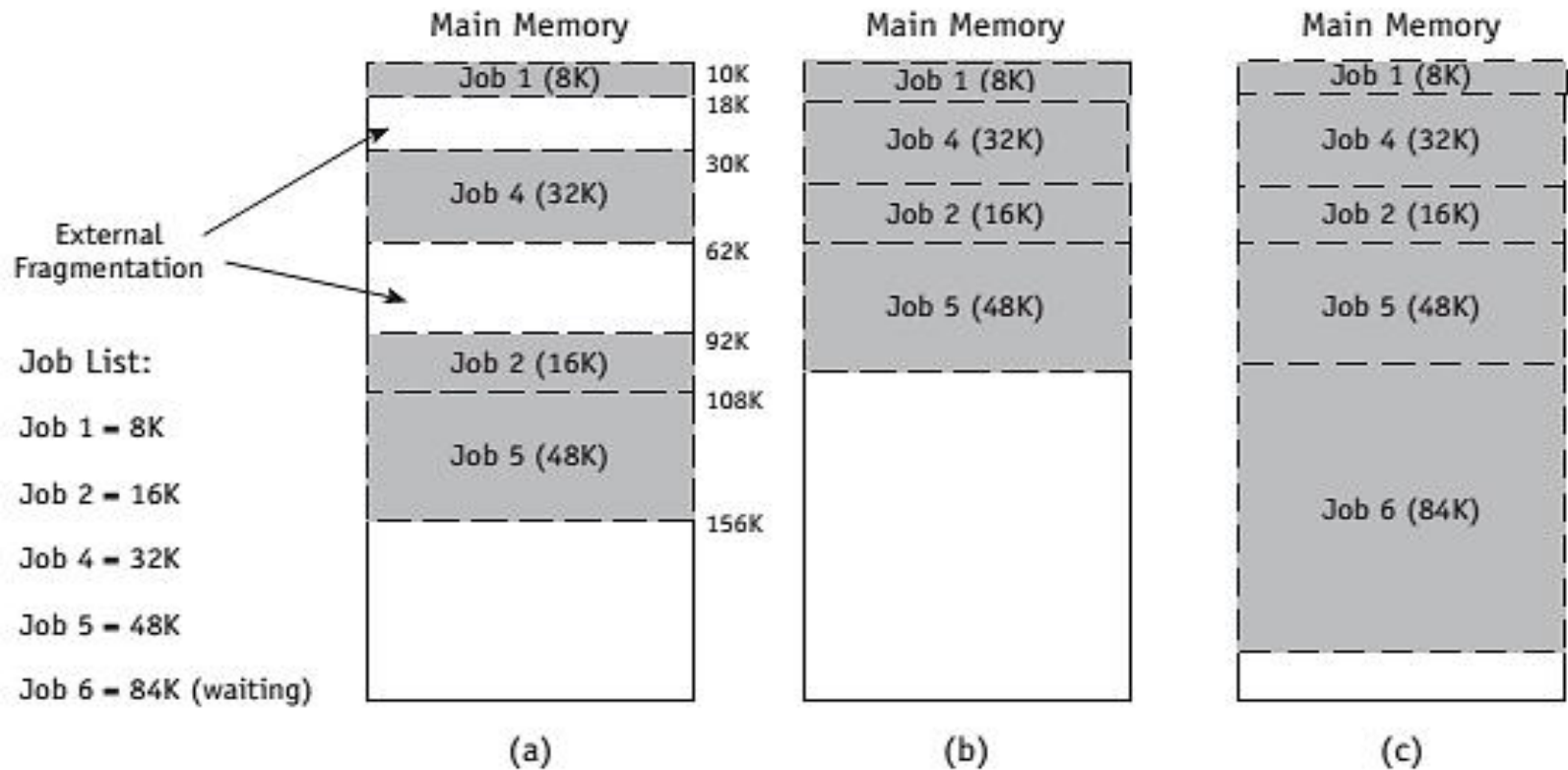
Compaction

- Compaction is a technique to collect all the free memory present in the form of fragments into one large chunk of free memory, which can be used to run other processes
- It does that by moving all the processes towards one end of the memory and all the available free space towards the other end of the memory so that it becomes contiguous
- It is not always easy to do compaction. Compaction can be done only when the relocation is dynamic and done at execution time.





Compaction





Non-Contiguous Memory Allocation

- **Non-Contiguous Memory Allocation-**
- Non-contiguous memory allocation is a memory allocation technique.
- It allows to store parts of a single process in a non-contiguous fashion.
- Thus, different parts of the same process can be stored at different places in the main memory.

- **Techniques-**
- There are two popular techniques used for non-contiguous memory allocation-
 - ✓ Paging
 - ✓ Segmentation





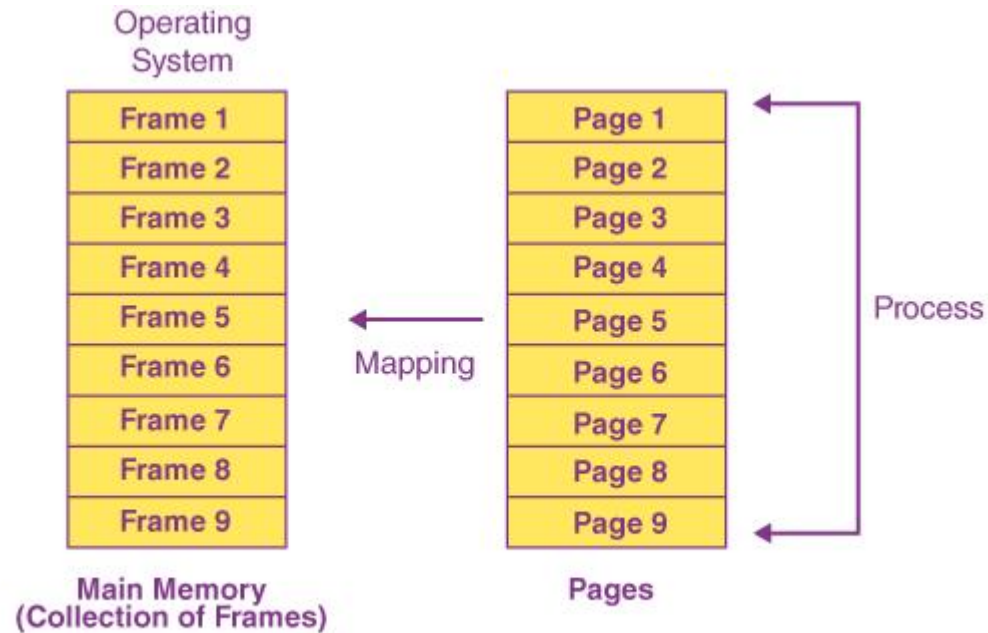
Paging

- **Paging**
- Paging is a fixed size partitioning scheme.
- In paging, secondary memory and main memory are divided into equal fixed size partitions.
- The partitions of logical address space are called as **pages**.
- The partitions of main memory are called as **frames**.
- One page of the process must be saved in one of the given memory frames. These pages can be stored in various memory locations.
- Process pages are usually only brought into the main memory when they are needed; else, they are stored in the secondary storage.





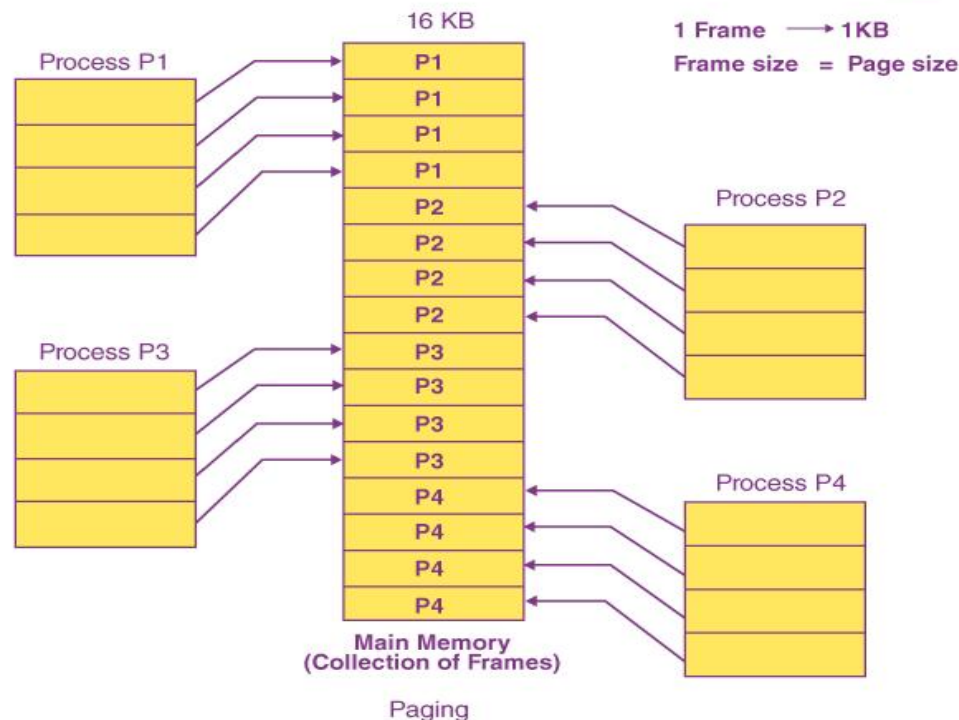
Paging





Paging

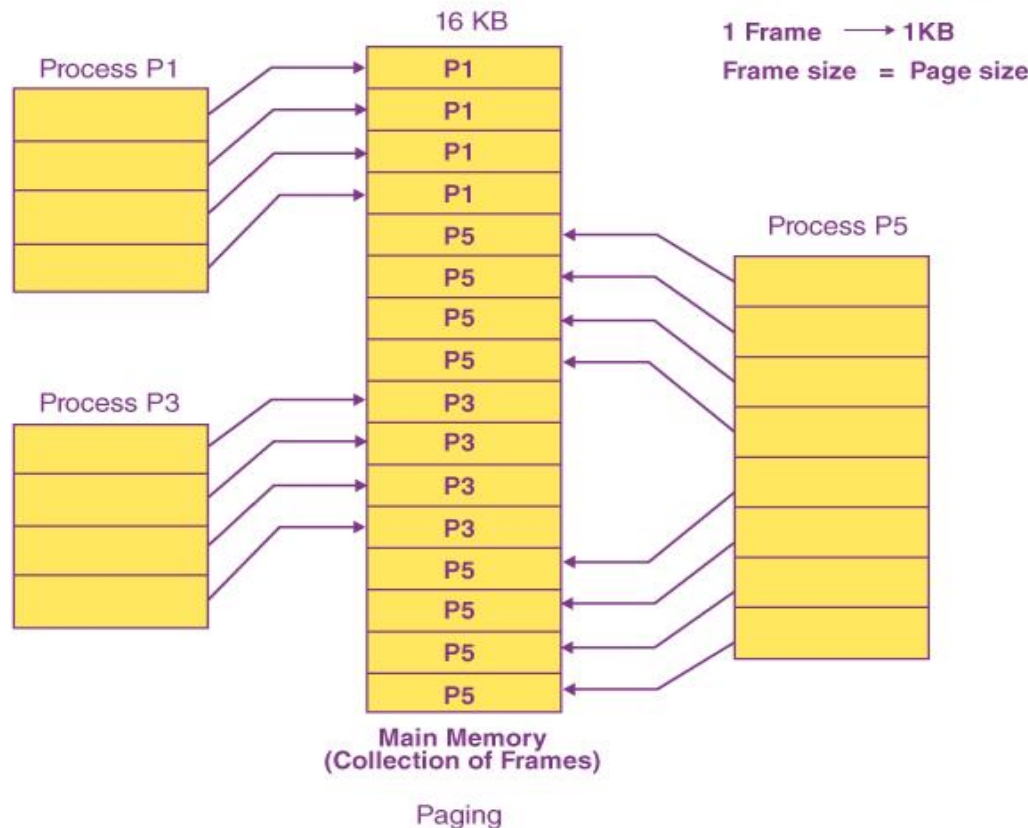
- Assuming that the main memory is 16 KB and the frame size is 1 KB, the main memory will be partitioned into a collection of 16 1 KB frames. P1, P2, P3, and P4 are the four processes in the system, each of which is 4 KB in size. Each process is separated into 1 KB pages, allowing one page to be saved in a single frame.





Paging

Suppose p2, p4 terminates and p5 from ready queue loaded into main memory

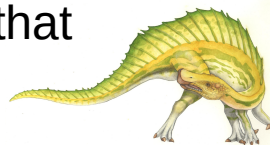


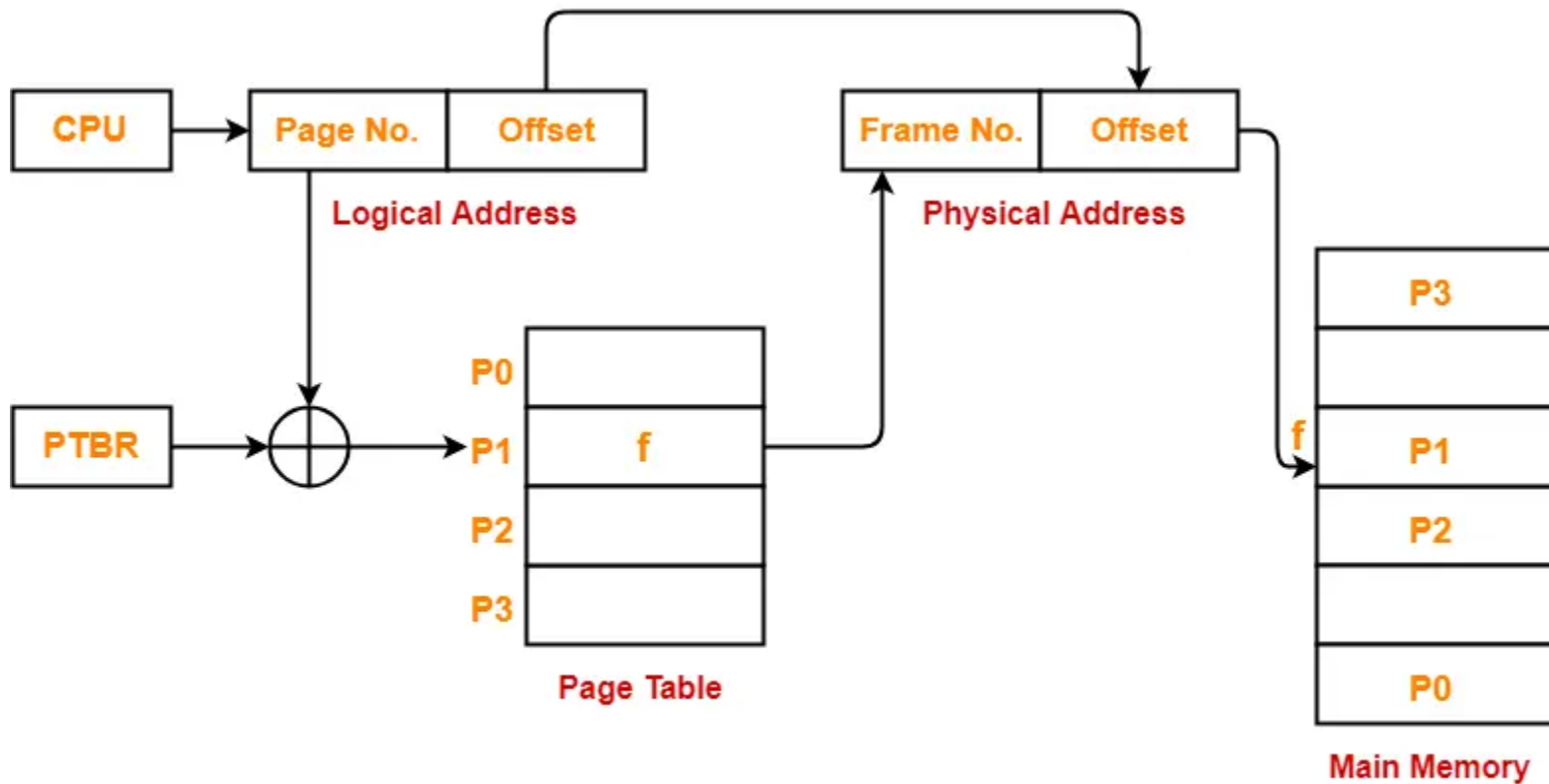


Address scheme mechanism

■ Address generated by CPU is divided into:

- **Page number (p)** – (eg: page 2) used as an index into a **page table** which contains base address of each page in physical memory or it specifies the specific page of the process from which CPU wants to read the data.
 - **Page offset (d)** – (eg: 10th instruction) combined with base address to define the physical memory address that is sent to the memory unit or specifies the specific word on the page that CPU wants to read.
- For the page number generated by the CPU,
 - **Page Table(page no, frame no)** provides the corresponding frame number (base address of the frame) where that page is stored in the main memory.
 - The frame number combined with the page offset forms the required physical address.
 - Frame number specifies the specific frame where the required page is stored.
 - Page Offset specifies the specific word that has to be read from that page.





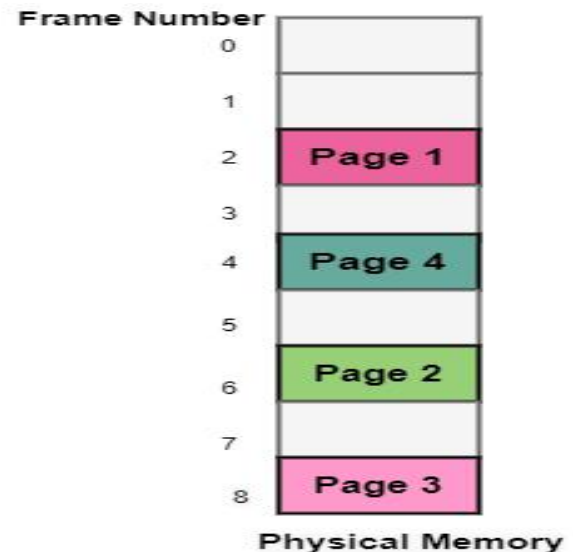
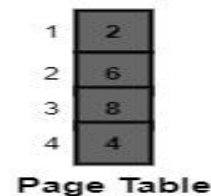
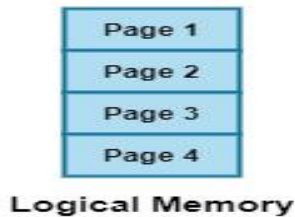
Translating Logical Address into Physical Address





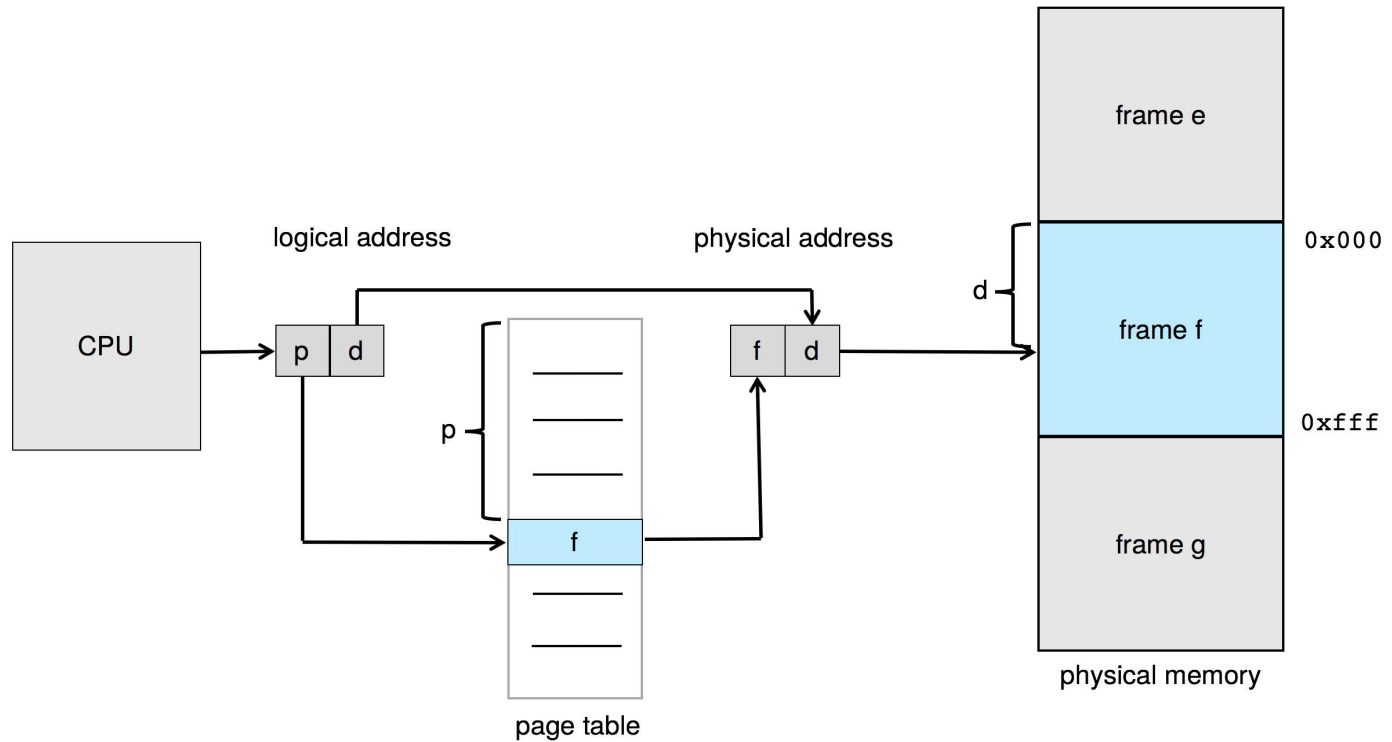
Page table

- Data structure that is used by the operating system of a computer in order to store the mapping between physical and logical addresses is commonly known as **Page Table**.
- Logical address that is generated by the CPU is translated into the physical address with the help of the page table.
- Page table mainly provides the corresponding frame number (base address of the frame) where that page is stored in the main memory.



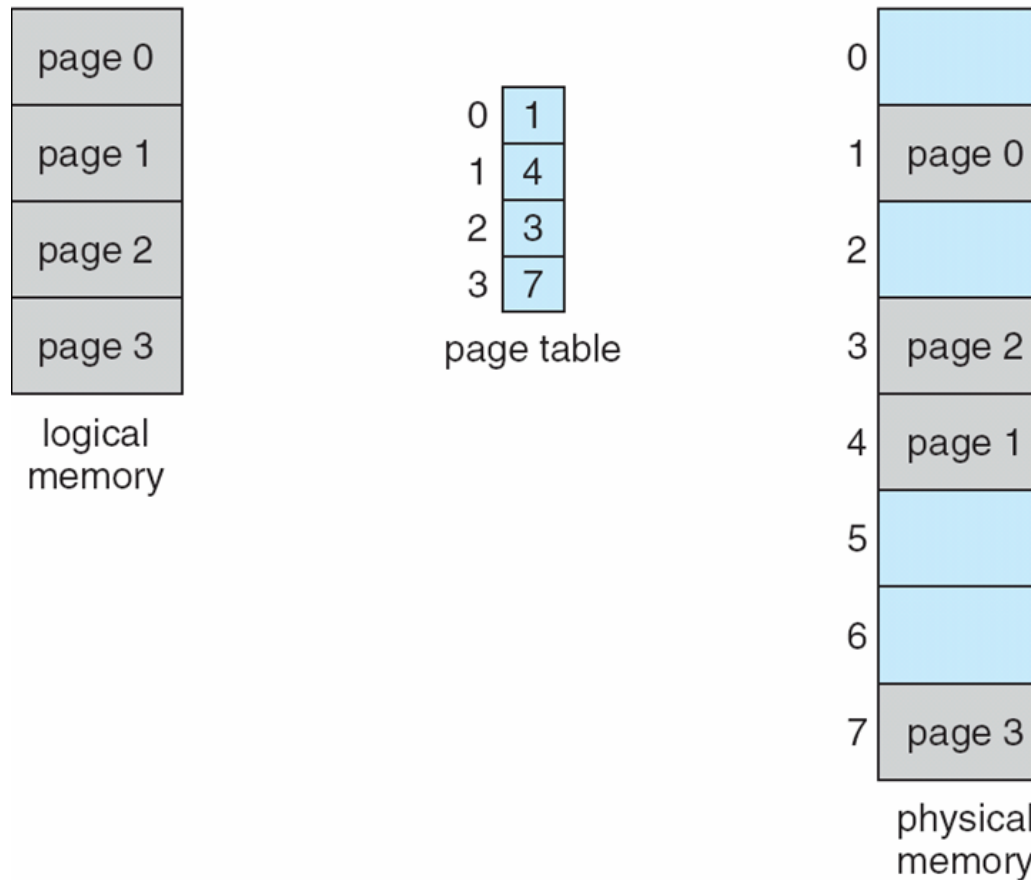


Paging Hardware





Paging Model of Logical and Physical Memory





Paging Example

- Logical address: Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages)

Process(P)
address space
with 4 pages
(P0,P1,P2,P3)

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Physical address = (frame no x page size) +
displacement/offset

Logical address 3 (page 0, offset 3)





Paging Example

Logical address 0 is page 0, offset 0. Indexing into the page table, we find that page 0 is in frame 5. Thus, logical address 0 maps to physical address 20 ($= (5 \times 4) + 0$).

Logical address 3 (page 0, offset 3) maps to physical address 23 ($= (5 \times 4) + 3$).

Logical address 4 is page 1, offset 0; according to the page table, page 1 is mapped to frame 6. Thus, logical address 4 maps to physical address 24 ($= (6 \times 4) + 0$).





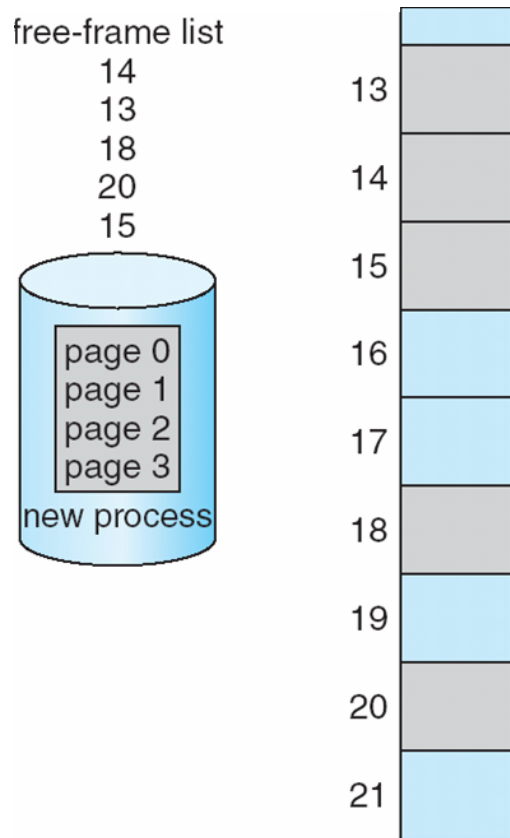
Paging -- Calculating internal fragmentation

- Page size = 2,048 bytes
- Process size = 72,766 bytes
- 35 pages + 1,086 bytes
- Internal fragmentation of $2,048 - 1,086 = 962$ bytes
- Worst case fragmentation = 1 frame – 1 byte
- On average fragmentation = $1 / 2$ frame size
- But each page table entry takes memory to track
- Page sizes growing over time



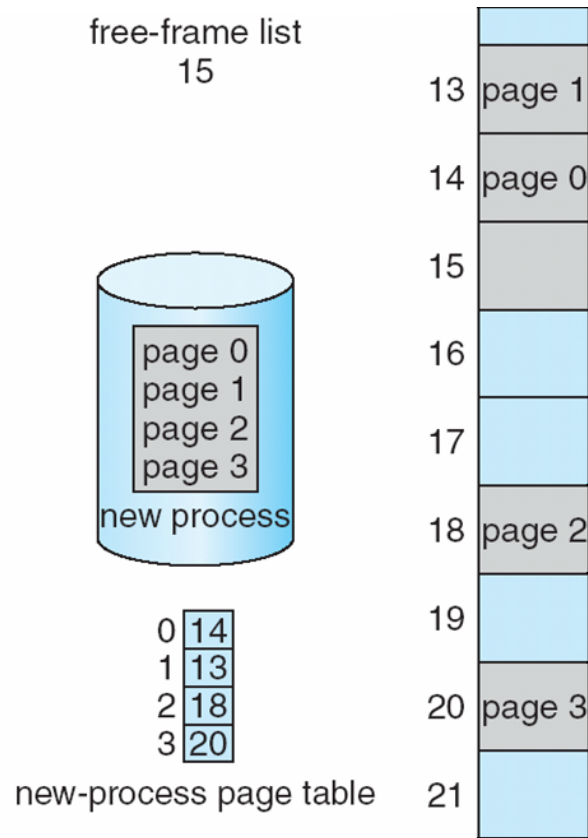


Free Frames



(a)

Before
allocation



(b)

After
allocation

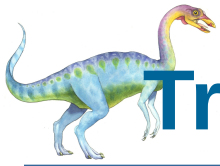




Implementation of Page Table

- Page table is kept in main memory
 - **Page-table base register (PTBR)** points to the page table
 - **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data / instruction
- The two-memory access problem can be solved by the use of a special fast-lookup hardware cache called **translation look-aside buffers (TLBs)** (also called **associative memory**).





Translation Look-Aside Buffer (TLB)

TLB is the cache for page table entries.

It reduces address translation time.

A TLB miss triggers a page table lookup, which is slower.

The TLB is associative, high-speed memory. Each entry in the TLB consists of two parts: a key (or tag) and a value.

When the associative memory is presented with an item, it is compared with all keys simultaneously.

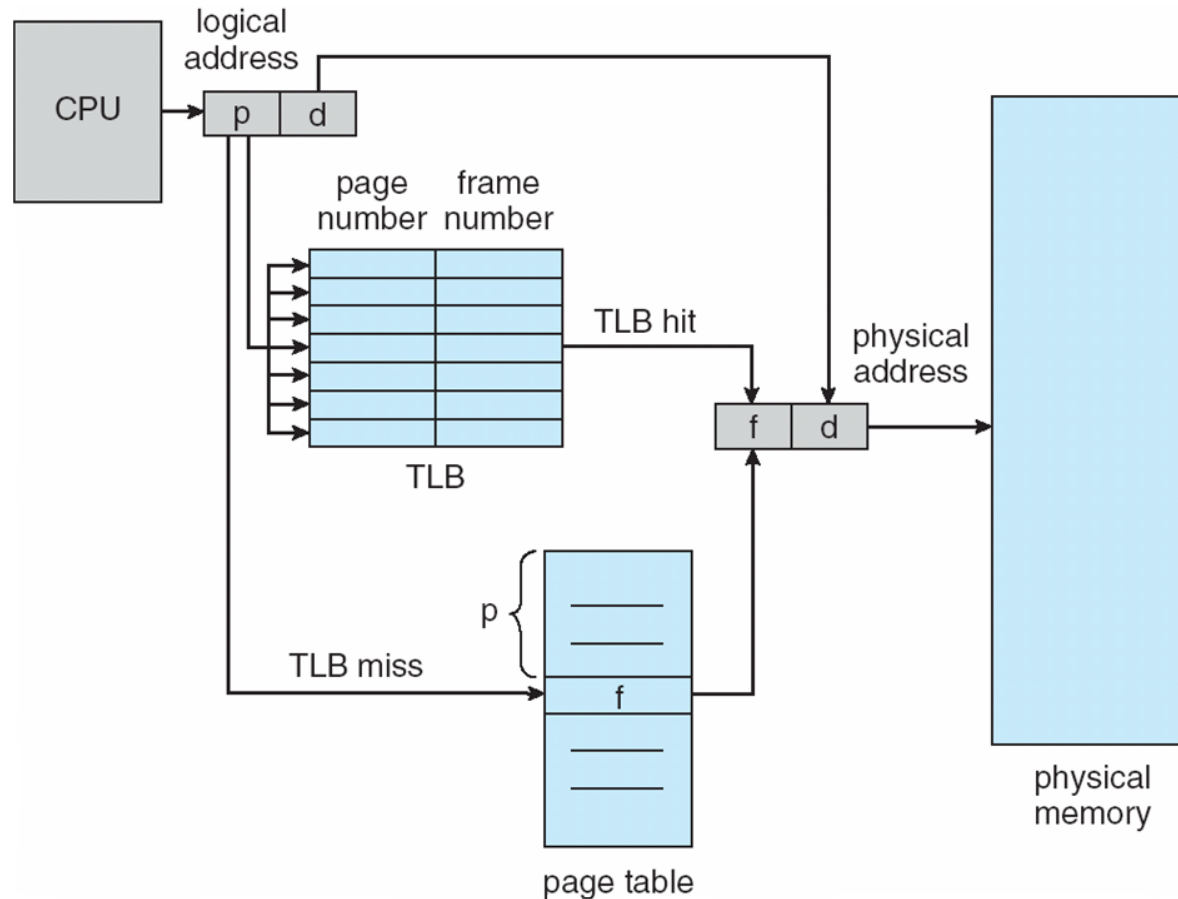
If the item is found, the corresponding value field is returned. The search is fast; the hardware, however, is expensive. Typically, the number of entries in a TLB is small, often numbering between 64 and 1,024.

- TLBs typically small (64 to 1,024 entries)
- On a TLB miss, value is loaded into the TLB for faster access next time





Paging Hardware With TLB





Memory Protection

Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed

- Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid** bit attached to each entry in the page table:
 - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “invalid” indicates that the page is not in the process’ logical address space
- Any violations result in a trap to the kernel





Valid (v) or Invalid (i) Bit In A Page Table

00000	page 0
	page 1
	page 2
	page 3
	page 4
10,468	page 5
12,287	

frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

0	
1	
2	page 0
3	page 1
4	page 2
5	
6	
7	page 3
8	page 4
9	page 5
	⋮
	page <i>n</i>





Shared Pages

■ Shared code

- One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
- Similar to multiple threads sharing the same process space
- Also useful for interprocess communication if sharing of read-write pages is allowed

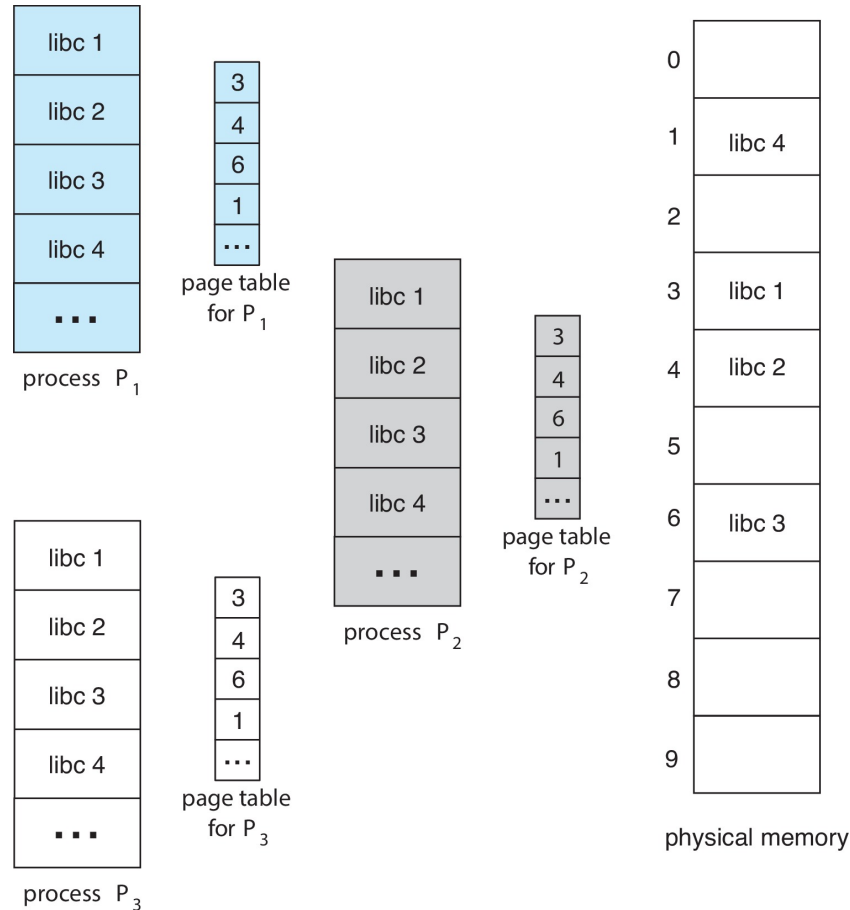
■ Private code and data

- Each process keeps a separate copy of the code and data
- The pages for the private code and data can appear anywhere in the logical address space





Shared Pages Example



End of Chapter 9

